1.5. Summary

Chapter 2. Managing threads

2.1. Basic thread management

2.1.1. Launching a thread

2.1.2. Waiting for a thread to complete

2.1.3. Waiting in exceptional circumstances

2.1.4. Running threads in the background

2.2. Passing arguments to a thread function

2.3. Transferring ownership of a thread

2.4. Choosing the number of threads at runtime

2.5. Identifying threads

2.6. Summary

Chapter 3. Sharing data between threads

3.1. Problems with sharing data between threads

3.1.1. Race conditions

3.1.2. Avoiding problematic race conditions

3.2. Protecting shared data with mutexes

3.2.1. Using mutexes in C++

3.2.2. Structuring code for protecting shared data

3.2.3. Spotting race conditions inherent in interfaces

3.2.4. Deadlock: the problem and a solution

3.2.5. Further guidelines for avoiding deadlock

3.2.6. Flexible locking with std::unique_lock

3.2.7. Transferring mutex ownership between scopes

3.2.8. Locking at an appropriate granularity